

Answer Key Evaluation DEP

Unified technical overview — one source file, one PDF

Project documentation deck

May 13, 2026

Outline (matches section order)

- ➊ **Introduction** — repository layout and system diagram.
- ➋ **Frontend** — React/Vite atlas and representative code.
- ➌ **Backend and APIs** — routers, REST surface, domain models, objective ingress paths.
- ➍ **Persistence and configuration** — SQLite, cache keys, Supabase pool, migrations, environment variables.
- ➎ **Authentication** — Supabase vs FastAPI boundary.
- ➏ **Gemini and Vertex** — how vision and text models are invoked (objective + code paths).
- ➐ **Objective grading pipeline (services)** — TikZ chain + module cheat-sheet (after Gemini overview).
- ➑ **Code evaluation** — C-from-image, gcc, syntax repair; then **LeetCode testcase library**.
- ➒ **Google Drive and Sheets** — integration snippets.
- ➓ **ZIP, PDF, Sheets publishing, and Evaluate UI** — multipart ZIP/PDF uploads, page splitting and synthetic IDs, master/marks/response/Super tabs, export endpoints, React evaluate page modes and toolbar.

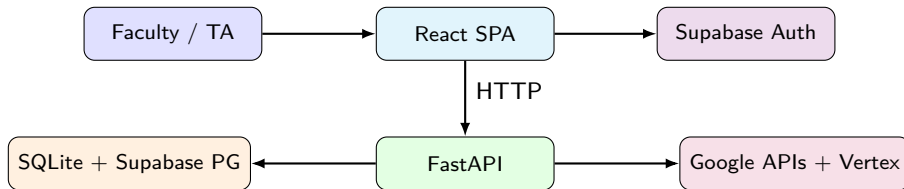
How to read this deck

- **Objective grading:** ZIP, Drive folder, or PDF → OCR → answer-key match → SQLite/cache → optional Sheets/Drive rename.
- **Code grading:** separate HTTP prefix `/api/code-eval/`; shares Gemini patterns but uses `c_code_evaluator` + LeetCode-shaped tests.
- Later slides follow section order: persistence/config → auth → Gemini → objective pipeline → code-eval (including LeetCode) → Drive/Sheets → ZIP/PDF + Sheets publishing + Evaluate UI → throughput/fuzzy/API contracts → OCR reference → supplemental coverage → closing.

Top-level repository layout

Path	Purpose
<code>frontend-new-design/</code>	Vite React SPA: Supabase Auth + UI; calls FastAPI via <code>VITE_BACKEND_URL</code> .
<code>backend/</code>	FastAPI <code>main.py</code> ; <code>routers</code> <code>api/endpoints.py</code> , <code>batch_endpoints_optimized.py</code> , <code>code_eval_endpoints.py</code> .
<code>backend/services/*.py</code>	Drive, Sheets, OCR variants, evaluation, cache, PDF split, DB pool, etc.
<code>backend/migrations/*.sql</code>	Supabase DDL for <code>result_cache</code> , <code>pdf_page_map</code> .
<code>c_code_evaluator/</code>	Image→C code (Gemini); <code>compile-fix</code> ; LeetCode harness (<code>leetcode_mode.py</code>).

End-to-end system context (single product)



Same PDF covers UI wiring (left), API routers (center), persistence (cache + SQLite), vision/compiler backends (right), and the optional code-eval path.

Frontend file atlas — entry & routing

File	Role
<code>src/index.jsx</code>	createRoot; mounts App; imports Tailwind/CSS.
<code>src/App.jsx</code>	Wraps AuthProvider around Routes.
<code>src/Routes.jsx</code>	BrowserRouter: public /login-register; protected /dashboard-overview, /faculty-dashboard, /course/:courseId, /evaluate/:evaluationId, /kanban-board, /task-detail, /analytics-dashboard.
<code>components/ProtectedRoute.jsx</code>	Spinner while useAuth loading; redirect if no user.
<code>components/ErrorBoundary.jsx</code>	Isolates render failures.
<code>components/ScrollToTop.jsx</code>	UX on navigation.
<code>components/ui/*</code>	Shared UI primitives (Header, Button, Input, modals).
<code>utils/cn.js</code>	Class-name merging helper.

Frontend file atlas — services

File	Role
<code>services/supabaseClient.js</code>	<code>createClient</code> ; <code>storageKey</code> per Supabase project ref; stale token cleanup.
<code>services/authService.js</code>	<code>signInWithPassword</code> , <code>signUp</code> , inserts profiles, friendly errors.
<code>services/backendService.js</code>	All FastAPI calls; <code>requireEvaluationId</code> ; multipart ZIP; batch URLs.
<code>services/evaluationService.js</code>	Supabase evaluations, duties, nested course/profile selects.
<code>services/resultsService.js</code>	Reads aggregated results for UI.
<code>services/courseService.js</code> , <code>studentService.js</code>	Course/student helpers.
<code>services/teamService.js</code> , <code>invitationService.js</code>	TA roster + invitations (used from course-detail, faculty-dashboard); Supabase-backed.

Frontend file atlas — pages (screens)

Area	Role
pages/login-register/* pages/dashboard-overview/*	Auth forms + social hooks. Landing tiles + quick actions; links target /sprint-planning, /team-management (see routing caveat).
pages/faculty-dashboard/* pages/course-detail/	Course hub + pending invitations. Course shell: Drive/Sheet URLs, evaluations, TA invites (teamService/invitationService).
pages/evaluate/index.jsx pages/kanban-board/*, pages/task-detail/*	Main grading: answer key, ZIP/Drive/PDF, export/sync/rename. Board + task detail shells.
pages/analytics-dashboard/ pages/sprint-planning/*, pages/team-management/*	Aggregate metrics. Code exists under src/pages/; not registered in Routes.jsx — dashboard/command palette links would hit catch-all redirect unless routes added.

Routes.jsx: mounted paths (authoritative)

```
<Route path="/login-register" element={<LoginRegister />} />
<Route path="/dashboard-overview" element={<ProtectedRoute><DashboardOverview /></ProtectedRoute>} />
<Route path="/faculty-dashboard" element={<ProtectedRoute><FacultyDashboard /></ProtectedRoute>} />
<Route path="/course/:courseId" element={<ProtectedRoute><CourseDetail /></ProtectedRoute>} />
<Route path="/evaluate/:evaluationId" element={<ProtectedRoute><EvaluatePage /></ProtectedRoute>} />
<Route path="/kanban-board" element={<ProtectedRoute><KanbanBoard /></ProtectedRoute>} />
<Route path="/task-detail" element={<ProtectedRoute><TaskDetail /></ProtectedRoute>} />
<Route path="/analytics-dashboard" element={<ProtectedRoute><AnalyticsDashboard /></ProtectedRoute>} />
```

Root / redirects to login; * → login. sprint-planning/ and team-management/ folders exist but have no Route here.

backendService.js: evaluation-scoped batch calls

```
const requireEvaluationId = (evaluationId, action) => {  
  if (!evaluationId || String(evaluationId).trim().toLowerCase() === 'default') {  
    throw new Error("Missing evaluationId for " + action);  
  }  
  return String(evaluationId).trim();  
};  
// processDriveFolder -> POST /api/batch/process-folder-optimized + evaluation_id
```

evaluate/index.jsx: answer key from Drive

```
const res = await backendService.extractAnswerKeyFromDrive(driveUrl);  
await evaluationService.saveAnswerKey(evaluation.id, res.answer_key);  
onKeyLoaded(res.answer_key);
```

Bootstrap: index.jsx and App.jsx

```
import { createRoot } from "react-dom/client";
createRoot(container).render(<App />);

import { AuthProvider } from "../context/AuthContext";
function App() {
  return (<AuthProvider><Routes /></AuthProvider>);
}
```

Dashboard overview and navigation affordances

dashboard-overview: metric-style landing + quick actions; navigates to sprint planning / team management paths *by URL only* — those routes are not declared in `Routes.jsx`.
`CommandPalette.jsx` registers keyboard-driven jumps to the same paths; useful after routes are wired.

Faculty dashboard and course detail

Faculty dashboard: course cards entry point; surfaces pending TA invitations via `invitationService`.

Course detail: binds Google Drive folder + master spreadsheet URLs to a course; evaluation list; TA search/add/remove and invite/cancel flows (`teamService`, `invitationService`).
Supabase rows drive persistence — FastAPI remains grading-only.

Kanban, task detail, analytics

Kanban board: column/card UI shell for duty-style tasks.

Task detail: attachments, comments, markdown notes, dependencies — PM metaphors adjacent to grading.

Analytics dashboard: read-mostly charts tied to evaluation throughput (expects backend results populated).

Supabase application schema (frontend-facing)

Not duplicated in backend/migrations SQL slides: React services expect tables such as profiles, courses, evaluations, duties/joins for TA invites. `evaluationService.saveAnswerKey` stores serialized keys per evaluation row; `courseService/studentService` wrap CRUD selectors. RLS and policies live in Supabase project — backend Postgres cache (`result_cache`) is a separate concern from app roster data.

Backend file atlas

Path	Role
<code>main.py</code>	FastAPI app; CORS; lifespan (Supabase pool, OCR cleanup); includes 3 routers.
<code>models.py</code>	Pydantic: <code>AnswerKey</code> , <code>StudentResult</code> , <code>ProcessFolderRequest</code> , etc.
<code>database.py</code>	SQLite CRUD (students, submissions, results).
<code>cli.py</code>	Headless: Drive → OCR (local or Gemini) → score → Sheets.
<code>api/endpoints.py</code>	Monolithic standard API (ZIP, Drive, Sheets, cache, pipeline runs, rename).
<code>api/batch_endpoints_optimized.py</code>	Optimized batch + PDF runs + cache purge + reconciliation APIs.
<code>api/code_eval_endpoints.py</code>	Code grading HTTP surface; Zip + Drive image ingestion.

/api/* endpoints (representative groups)

- /status, /performance
- /answer-key (+ extract/upload/set-manual)
- /process-zip, /process-zip/start
- /process-drive-folder
- /export-to-sheets, /export-student-responses

- /full-pipeline*, /sync-*
- /results, /cache/*, /database/optimize
- /pipeline-runs/{id}/*
- /rename-drive-files*
- /download-answer-sheet-template

/api/batch/* and /api/pdf-runs/*

- POST /batch/process-folder-optimized (+ /start) — Drive throughput grading.
- GET /batch/processing-status/{id}, /batch/ocr-debug/{id}
- DELETE /cache/purge-evaluation/{evaluation_id}
- POST /batch/process-pdf-optimized* — PDF ingress.
- GET /pdf-runs/{id}/mapping, /mapping.csv, per-page image routes.
- POST /pdf-runs/{id}/reconcile
- Problem-image download helpers for QA.

/api/status vs /api/performance

GET /status: operational heartbeat + `answer_key_loaded`, in-memory results count, cache stats, SQLite analytics snapshot, runtime env flags.

GET /performance: same cache/DB analytics plus `optimization_recommendations` heuristics (cache size, pool hit ratio, batch ratio nudges).

/api/sync-drive and /api/scan-drive-folder

POST /sync-drive: query `folder_id` — legacy list-files only (images), returns raw file metadata count.

POST /scan-drive-folder: body `ProcessFolderRequest`; lists folder then `separate_files` into answer-key candidates vs student sheets — dry inspection before OCR spend.

Answer key REST surface (/api/answer-key*)

GET /answer-key — current in-memory key snapshot.

POST /answer-key/extract-from-drive — download + AnswerKeyService.extract_answer_key.

POST /answer-key/upload — multipart file upload path.

POST /answer-key/set-manual — JSON body installs key without file ingest.

full-pipeline: FullPipelineRequest (drive_folder_url, sheets_url) → awaits process_drive_folder then best-effort export_to_sheets; surfaces sheet_export_ok / error string without masking Drive success.

full-pipeline-with-rename: extends flow with Drive rename policy after grading (see endpoint docstring).

/api/sync-results

POST body: master sheet_url, optional results payload + subsheet_name.

Delegates to SheetsService.sync_results_with_master — reconciles OCR rows with roster ordering / merged marks after manual Sheet edits.

/api/results, cache hygiene, /api/database/optimize

GET /results: prefers session `_current_results`; falls back to SQLite legacy rows.

DELETE /results/clear: wipes session key + results.

POST /cache/cleanup|clear: LRU / destructive clears with guards.

POST /database/optimize: VACUUM, index repair via `optimized_db`; returns analytics snapshot.

`/api/processing/status` and pipeline run APIs

Legacy single-flight processing snapshot (distinct from batch `/batch/processing-status/{id}`).
`/pipeline-runs/{run_id}` (+ `/status`, `/items`) expose SQLite-backed ZIP/async runs for resume UX.

/api/process-file (legacy gate)

Disabled unless `ENABLE_LEGACY_PROCESS_FILE=1`. Otherwise HTTP 410 directing callers to Drive/ZIP/batch paths.

When enabled: downloads one Drive file, uses legacy `ocr_service.extract_data`, writes `submissions/results` — divergent schema from modern `StudentResult` pipeline.

Answer sheet template download

GET /download-answer-sheet-template? format=csv|xlsx|json & num_questions — emits blank structures describing SMCQ/MMCQ/NCQ fields for print workflows.

Objective ingress: ZIP, Drive, PDF

Three surfaces share `match_and_score` / cache semantics; differ only in how bytes reach FastAPI.

ZIP (/api/process-zip)

Multipart upload → extract images → optional answer key in archive → OCR → scoring → SQLite/cache → pipeline run_id for polling.

Drive (/batch/process-folder-optimized)

Folder ID → list/download → cache lookup `sheet:v2:{eval}:{file_id}` → dual Pro OCR + tie-break → batch writes → optional inline Drive rename.

PDF (/batch/process-pdf-optimized)

pdf_split_service renders pages → synthetic IDs pdf:<sha>:pN → same OCR+eval as ZIP → pdf_page_map + reconcile APIs.

Pydantic contracts (models.py)

```
class AnswerKeyEntry(BaseModel):
    question_type: str
    correct_answer: str
    positive_marks: float = 1.0
    negative_marks: float = 0.0

class StudentResult(BaseModel):
    entry_number: str
    name: str
    total_score: float
    max_score: float
    details: List[QuestionResult] = []
```

Requests (models.py)

```
class ProcessFolderRequest(BaseModel):
    folder_url: str
    evaluation_id: Optional[str] = None
    rename_drive_inline: bool = False
    group_multiple_pages: bool = False

class PipelineSummary(BaseModel):
    model_config = {"arbitrary_types_allowed": True}
    total_students_processed: int
    answer_key_source: str
    errors: List[Dict] = []
    processing_stats: Dict = {}
```

How objective routers relate (single backend)

`endpoints.py`: interactive ZIP/Drive/Sheets/templates/cache.

`batch_endpoints_optimized.py`: throughput + PDF world + purge.

`code_eval_endpoints.py`: separate C+LeetCode path — shares Drive download utilities only.

FastAPI composition (main.py)

```
app.include_router(endpoints.router)
app.include_router(batch_endpoints_optimized.router)
app.include_router(code_eval_endpoints.router)
```

Lifespan: warm asyncpg pool when `CACHE_BACKEND=supabase`; shutdown closes OCR clients + pool.

SQLite schema (database.py)

```
CREATE TABLE IF NOT EXISTS students
  (id TEXT PRIMARY KEY, name TEXT, roll_number TEXT)
CREATE TABLE IF NOT EXISTS submissions
  (id TEXT PRIMARY KEY, student_id TEXT, exam_id TEXT,
   file_id TEXT, status TEXT, extracted_data TEXT)
CREATE TABLE IF NOT EXISTS results
  (submission_id TEXT PRIMARY KEY, score REAL,
   feedback TEXT, details TEXT)
```

SQLite pipeline runs (optimized_database_service.py)

Beyond database.py legacy tables, WAL pool creates:

pipeline_runs: run_id, source_type (drive|zip|manual), source_ref, status counters, JSON errors.

pipeline_run_items: per-file status, optional ocr_json / result_json for resume & QA.

Local pdf_page_map table mirrors Supabase PDF mapping when runs originate from PDF splitter (same reconciliation semantics).

Sheet cache key (result_cache_service.py)

```
def sheet_cache_key(evaluation_id: str, file_id: str) -> str:  
    return f"sheet:v2:{evaluation_id}:{file_id}"
```

One row holds OCR JSON + optional scored result; answer_key_hash detects stale recompute when key edits.

SQLite WAL tuning (optimized_database_service.py)

```
conn.execute("PRAGMA_journal_mode=WAL")  
conn.execute("PRAGMA_synchronous=NORMAL")  
conn.execute("PRAGMA_mmap_size=268435456")
```

Supabase DDL (full excerpts later)

Row-level `result_cache JSONB` lives in Postgres when `CACHE_BACKEND=supabase`; complete `CREATE TABLE` and scope migration slides appear in **OCR and migrations reference**.

Supabase async pool (supabase_client.py)

```
1 _pool = await asyncpg.create_pool(  
2     dsn=os.getenv("SUPABASE_DB_URL"),  
3     min_size=int(os.getenv("SUPABASE_POOL_MIN", "1")),  
4     max_size=int(os.getenv("SUPABASE_POOL_MAX", "10")),  
5     statement_cache_size=0, # PgBouncer-friendly  
6 )
```

Environment variables (checklist)

Var	Role
VITE_BACKEND_URL, VITE_SUPABASE_URL, VITE_SUPABASE_ANON_KEY	Frontend
GOOGLE_APPLICATION_CREDENTIALS, GOOGLE_CLOUD_PROJECT	Vertex + Drive/Sheets SA
GOOGLE_API_KEY / GEMINI_API_KEY	AI Studio REST fallback
GEMINI_OCR_MODEL, VERTEX_LOCATION	Code-eval header OCR + C OCR defaults
CACHE_BACKEND, SUPABASE_DB_URL	Cache backend
CORS_ORIGINS	Browser origins

Auth model (two planes)

Browser ↔ Supabase Email/password (or social) via @supabase/supabase-js. JWT/session stored under project-scoped storageKey. Rows in profiles, evaluations, courses — **not** validated by FastAPI today.

Browser ↔ FastAPI Plain fetch to /api/*; no Supabase JWT verification on backend for most routes — backend is “helpers + grading engines” behind your deployment perimeter.

Backend ↔ Google Service account JSON (GOOGLE_APPLICATION_CREDENTIALS) or API keys for Gemini REST.

Backend ↔ Supabase Postgres Optional SUPABASE_DB_URL when CACHE_BACKEND=supabase — DB role bypasses RLS.

Frontend: Supabase client + AuthProvider pattern

```
// supabaseClient.js - project ref isolates tokens
authOptions.storageKey = 'sb-' + PROJECT_REF + '-auth-token';

// App.jsx
<AuthProvider><Routes /></AuthProvider>

// ProtectedRoute.jsx - gate on useAuth().user
if (!user) return <Navigate to="/login-register" replace />;
```

Auth extrapolation

- Attach Supabase JWT to FastAPI (Authorization: Bearer) + verify with JWKS / shared secret.
- Map `profiles.role` to backend RBAC (who may purge cache, rename Drive, export Sheets).
- Audit log table keyed by Supabase auth subject id for institutional compliance.

Gemini / Vertex: unified picture

Consumer	Transport	Typical env
OCRService, AnswerKeyService	Vertex SA bearer <i>or</i> AI Studio key URL	GOOGLE_APPLICATION_CREDENTIALS, GOOGLE_API_KEY
OptimizedOCRService / multi-region	Vertex regional generateContent	SA JSON per re- gion under sa-keys/ GOOGLE_CLOUD_PROJECT
Optimized batch pipeline evaluate_c_from_image.py	Policy: Gemini 2.5 Pro multi-region Vertex stream <i>then</i> fallback generativelanguage + key	Same SA pool; rate limiting internal GEMINI_API_KEY, VERTEX_LOCATION
code_eval_endpoints.py student header OCR	_vertex_stream_generate_content or REST key	GEMINI_OCR_MODEL default flash- lite
Syntax-fix agent	_vertex_generate_content _gemini_generate_content	or temperature 0

Why multiple Gemini “clients” in one repo?

- **Latency vs quality:** Flash/Lite for cheap/high-volume probes; Pro for disagreements in batch policy.
- **Auth flexibility:** On-prem SA JSON for Vertex; student laptops may only have AI Studio API keys for coursework demos.
- **Streaming vs non-stream:** Vision OCR uses stream aggregator; syntax-fix uses single-shot generate to simplify parsing.
- **Isolation:** Code-eval boots optional `vertex_key.json` and extends import path only inside `code_eval_endpoints`, leaving objective OCR configuration untouched.

Vertex streaming URL shape (evaluate_c_from_image.py)

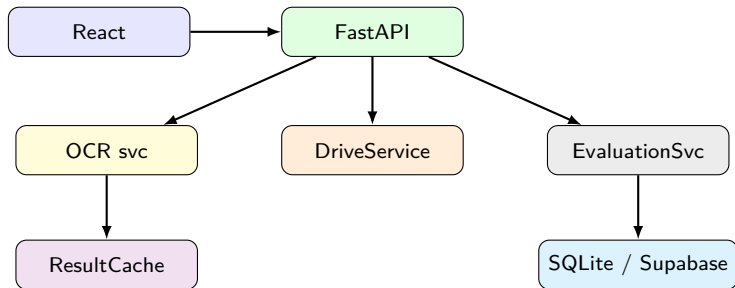
```
url = (  
    f"https://{location}-aiplatform.googleapis.com/v1/"  
    f"projects/{project_id}/locations/{location}/"  
    f"publishers/google/models/{model}:streamGenerateContent "  
)  
headers = {"Authorization": f"Bearer_{creds.token}", ...}
```

AI Studio fallback: <https://generativelanguage.googleapis.com/v1beta/models/{model}:streamGenerateContent?key=...>

Rate limits: Gemini REST retries (`_gemini_stream_generate_content`)

```
for attempt in range(max_retries):  
    resp = requests.post(url, json=payload, timeout=timeout_s)  
    if resp.status_code == 429:  
        wait = 2 ** attempt * 5  
        time.sleep(wait)  
        continue
```

Objective pipeline: responsibility chain



Batch optimized path: download bytes → preprocess → Gemini OCR (dual Pro + vote) → `match_and_score` → `cache row sheet:v2:{eval}:{file}` → optional Drive rename + Sheets export.

Service cheat-sheet (objective)

Module	Behavior
drive_service.py	OAuth SA or API key; list/download; batch rename.
pdf_split_service.py	PDF → page JPEGs; pdf:<sha>:pN synthetic IDs for cache stability.
image_preprocessing.py	Paper crop + JPEG sizing before model POST.
ocr_service.py	Legacy single-image Gemini Flash stream.
optimized_ocr_service.py /	Pooled HTTP; load-aware endpoint pick.
multi_region_ocr_service.py	
answer_key_service.py	Parse key files / vision-parse via Gemini.
evaluation_service.py	Pure match_and_score; SMCQ/MMCQ/NCQ + negatives.
batch_evaluation_service.py	Parallel scoring over many OCR dicts; stats.
result_cache_service.py	SQLite or Supabase; LRU; evaluation-scoped keys.
optimized_database_service.py	WAL SQLite pool; batch inserts.
sheets_service.py	Match entry regex; write totals / per-Q / comments.
drive_rename_service.py	Filename policy after grades (FUZZY-, UNMATCHED-, etc.).

Code evaluator: purpose

Grade **handwritten or photographed C solutions** per question using:

- Gemini/Vertex **vision OCR** to reconstruct C source from an image.
- **gcc** compile (-c object in FastAPI route; full link in leetcode_mode harness).
- Optional LLM **syntax repair** (max 2 attempts) with strict `validate_trivial_fix` guardrails.
- **LeetCode-shaped test cases**: fetched via `leetcode_mode` resolver `resolve_leetcode_problem_and_cases`, then executed in generated main harness per case.

Evaluator bootstrap (code_eval_endpoints.py)

```
_vertex_key_path = str(Path(...) / "vertex_key.json")
if os.path.exists(_vertex_key_path):
    os.environ["GOOGLE_APPLICATION_CREDENTIALS"] = _vertex_key_path
    os.environ["GOOGLE_CLOUD_PROJECT"] = ... # from JSON project_id

sys.path.insert(0, ... / "c_code_evaluator")
from evaluate_c_from_image import extract_c_code_from_image, ...
from leetcode_mode import resolve_leetcode_problem_and_cases, evaluate_leetcode_cases
```

Sets GOOGLE_CLOUD_PROJECT before importing modules that read it at import time.

HTTP surface (code_eval_endpoints.py)

```
router = APIRouter(prefix="/api/code-eval", tags=["code-evaluation"])

@router.post("/process-zip", response_model=CodeEvalResult)
async def process_zip_code_eval(file: UploadFile, answer_key: str = Form(")):
    # unzip images -> asyncio.to_thread(_process_single_image, ...)

@router.post("/process-drive-folder", response_model=CodeEvalResult)
async def process_drive_code_eval(request: DriveCodeEvalRequest):
    # DriveService.list_all_files_in_folder -> download -> same core processor
```

Answer key JSON for code eval

```
answer_key = {  
  "problems": {  
    "1": { "slug": "two-sum", "marks": 2 },  
    "2": { "slug": "valid-parentheses", "marks": 1 }  
  }  
}
```

slug is LeetCode titleSlug. Marks scale all-or-nothing per problem: all hidden examples must pass.

_process_single_image: step-by-step

- 1 **Student identity:** Gemini vision JSON extract (name, entry_number) with temperature 0; fallback regex from filename stem.
- 2 **Code OCR:** call `extract_c_code_from_image` with image path and API key — Vertex first if SA plus project, else API key.
- 3 **Sanitize:** drop header lines matching Name / Entry Number markers from OCR noise.
- 4 **Headers:** inject std includes if none present.
- 5 **Compile:** run GCC compile-only pass (`-c, -std=c11`).
- 6 **Repair loop:** up to two attempts of `fix_ocr_syntax_errors` guarded by `validate_trivial_fix`.
- 7 **Per problem:** `resolve_leetcode_problem_and_cases` then `evaluate_leetcode_cases`.
- 8 **Score:** full marks only when every fetched testcase passes for that slug.

Student metadata micro-OCR (code_eval_endpoints.py)

```
prompt = (  
    "Extract ONLY Name and Entry Number..."  
    "Return ONLY JSON with keys name and entry_number"  
)  
payload = {  
    "contents": [{"role": "user", "parts": [  
        {"text": prompt},  
        {"inline_data": {"mime_type": mime_type, "data": base64_image}},  
    ]}],  
    "generationConfig": {"temperature": 0.0, "maxOutputTokens": 256},  
}  
model = os.environ.get("GEMINI_OCR_MODEL", "gemini-2.5-flash-lite")
```

extract_c_code_from_image (evaluate_c_from_image.py)

```
prompt = (  
    "You are doing OCR on a photo of handwritten/printed C source code.\n"  
    "Return ONLY the reconstructed C code as plain text.\n"  
    "- Do NOT add explanations.\n"  
    "- Preserve newlines and indentation.\n"  
)  
payload = {  
    "generationConfig": {"temperature": 0.0, "maxOutputTokens": 2048},  
}  
# Vertex stream if GOOGLE_APPLICATION_CREDENTIALS + project else GEMINI_API_KEY  
return _strip_markdown_fences(raw)
```

fix_ocr_syntax_errors: LLM repair contract

```
prompt = (  
    "Fix ONLY trivial OCR-induced syntax errors..."  
    "Do NOT change logic, algorithms, or control flow.\n"  
    "ONLY fix: semicolons, brackets, OCR confusions 0/0, missing #include...\n"  
    "Return ONLY the corrected C code.\n"  
)  
# Uses _vertex_generate_content or _gemini_generate_content (non-stream)
```

validate_trivial_fix: safety rails

```
# Reject if >20 lines changed (SequenceMatcher opcodes)  
# Reject if for/while/switch/goto/do counts change  
# Reject disallowed new #includes (only stdlib whitelist)  
# Reject if operator token sequence drifts beyond small edit budget  
# Cap identifier rename drift (edit distance) bulk violations  
return True, "accepted"
```

Prevents “fix” pass from silently rewriting algorithms while repairing OCR typos.

ZIP ingestion implementation (process_zip_code_eval)

```
with tempfile.TemporaryDirectory() as tmpdir:
    zip_path.write_bytes(await file.read())
    zf.extractall(extract_dir)
    image_files = sorted([p for p in extract_dir.rglob("*")
                          if p.suffix.lower() in image_exts])
    for img in image_files:
        result = await asyncio.to_thread(
            _process_single_image, img, problems, work_dir, api_key)
```

`asyncio.to_thread` keeps FastAPI event loop responsive while CPU/gcc/Gemini-sync code runs in worker threads.

_compile_c in code-eval (syntax check)

```
proc = subprocess.run(  
    ["gcc", "-c", str(source_path), "-O2", "-std=c11", "-Wall", "-Wextra",  
     "-o", str(binary_path)],  
    capture_output=True, text=True, timeout=timeout_s,  
)
```

Object file proves compilation; actual LeetCode grading uses separate full-program harness compile (leetcode_mode._compile_harness) linking generated main.

Response model (CodeEvalResult)

```
class CodeEvalResult(BaseModel):  
    total_students_processed: int  
    results: List[Dict[str, Any]]  
    errors: List[str]
```

Each result dict mirrors objective-sheet scoring fields: `total_score`, `details` with per-Q testcase counts, `comments` joining problem notes.

Dual gcc roles (why two compile paths?)

- **Endpoint _compile_c**: fast sanity check student OCR blob compiles as translation unit before wasting network on testcase fetch.
- **Harness compile**: each testcase wraps student functions + typed helpers + `main`; requires link-time symbols (`ListNode`, etc.).
- If sanity compile fails, repair agent runs; harness compile may still fail for unrelated reasons (unsupported signature).

resolve_leetcode_problem_and_cases: merge strategy

- 1 Optional **JSON file**: `_parse_cases_file` — embed meta + explicit testcases for air-gapped runs.
- 2 Try **leetscrape family**: dynamic import `leetscrape` / `leetcode_scrape` / `leetscraper`; probe `get_problem`, `fetch_problem`, etc.
- 3 Try **pyleet**: `TestCaseRetriever` plus `get_testcase` helpers.
- 4 If still empty and slug set: **LeetCode GraphQL** `questionData` query — parse content HTML for Input/Output pairs + `exampleTestcases` fallback.
- 5 Drop cases without expected; `_dedupe_cases`; truncate to `max_cases`.

GraphQL probe (_fetch_from_graphql)

```
requests.post(  
    "https://leetcode.com/graphql",  
    json={"query": query, "variables": {"titleSlug": slug}},  
    timeout=timeout_s,  
)  
  
# question.metadata JSON -> LeetProblemSpec (function name, param types)  
# content HTML -> Input:/Output: regex pairs -> LeetCode args + expected
```

Network dependency: institution firewall may block; use cases_file fallback for exams.

Datamodels (leetcode_mode.py)

Type	Meaning
LeetProblemSpec	function_name, param specs (name + logical type string), return type, sources list.
LeetCodeCase	args list (Python literals after parsing), expected value, source provenance tag.
CFunctionSpec	Parsed C function signature from student OCR code via regex over bodies.

evaluate_leetcode_cases: execution model

- ① `extract_c_function_specs` on OCR code — find top-level functions; exclude `main`.
- ② `_select_function`: optional override, else match `problem.function_name`, else single candidate / last resort heuristic with notes.
- ③ For each case: `_render_case_harness` builds C file with helper structs for `ListNode/TreeNode`, binds args by type (`int`, arrays, `char*`, 2D grids), adds `main` that prints normalized output.
- ④ `gcc` full link harness → run binary → compare normalized stdout to expected string from `_serialize_expected`.
- ⑤ Aggregate `PASSED` / `WRONG_OUTPUT` / `TIMEOUT` / `COMPILE_ERROR` / `HARNESS_BUILD_ERROR`.

Harness selection excerpt (evaluate_leetcode_cases)

```
1 specs = extract_c_function_specs(code)
2 selected, selection_notes = _select_function(
3     specs,
4     override_name=function_name_override,
5     preferred_name=problem.function_name,
6 )
7 harness_code, expected_normalized = _render_case_harness(
8     code=code, func=selected, problem=problem, case=case)
9 _compile_harness(harness_path, harness_bin, timeout_s=compile_timeout_s)
```

Optional Python deps (grading machine)

- **pyleet** — programmatic official-style testcase tuples.
- **beautifulsoup4** — cleaner HTML strip for GraphQL content parsing (regex fallback if missing).
- **leetscrape** ecosystem (optional; multiple module names tried).
- **gcc** system compiler — required for any LeetCode harness run.

resolve_leetcode_problem_and_cases signature

```
def resolve_leetcode_problem_and_cases(  
    *, slug: Optional[str], question_id: Optional[str],  
    cases_file: Optional[Path], max_cases: int, timeout_s: int,  
) -> Tuple[LeetProblemSpec, List[LeetCase], List[str]]:  
    ...
```

Returns merged problem spec (function name + param metadata), deduped cases with expectations, and diagnostic notes strings from each fetcher.

JSON testcase file format (_parse_cases_file)

```
{
  "title_slug": "two-sum",
  "meta": { "name": "twoSum", "params": [...], "return": {...} },
  "testcases": [
    { "args": [[2,7,11,15], 9], "expected": [0, 1] },
    { "input": "...", "output": "..." }
  ]
}
```

pyleet path (_fetch_with_pyleet)

```
1 from pyleet import get_testcase
2 from pyleet.testcase_retriever import TestCaseRetriever
3
4 retriever = TestCaseRetriever()
5 question_data = retriever.get_question_data(resolved_slug)
6 meta = json.loads(question_data.get("metaData") or "{}")
7 pyleet_cases = get_testcase(title_slug=resolved_slug)
```

Builds LeetCode objects by adapting tuple inputs to arg lists based on parameter count inference.

leetscrape dynamic import (`_fetch_with_leetscrape`)

```
for module_name in ("leetscrape", "leetcode_scrape", "leetscraper"):
    module = importlib.import_module(module_name)
    # Try get_problem / fetch_problem / get_question ... with slug kwargs
```

Wide compatibility shim because PyPI package APIs differ; logs which module answered.

`_parse_example_pairs_from_content` (GraphQL HTML)

Regex scans stripped problem statement text for alternating `Input:` / `Output:` blocks; truncates at `Explanation:`.

Fallback: split `exampleTestcases` / `sampleTestCase` newline blocks into synthetic cases (may lack expected $\rightarrow$ dropped later).

Type-directed harness binding (`_build_input_binding`)

Maps Python literals from LeetCode cases to C declarations: `int` scalars, `bool`, C strings (`char buf[]`), `int` arrays (`int arr[] = {...}`), `char**` ragged strings, 2D grids with companion `colSizes`, linked lists via hidden `__build_listnode`, trees via `__build_tree` level-order arrays with null flags.

Printed output conventions

Integer scalars: `printf` with width-aware format (`%11d` for long long).

Booleans: strings `true/false`.

Arrays: bracketed comma-separated numbers.

ListNode/TreeNode: bracketed level-order / adjacency walk matching LeetCode aesthetics for string compare.

Case verdict aggregation (evaluate_leetcode_cases)

```
1 matched = case_run_ok and (not case_timed_out) and (  
2     actual_normalized == expected_normalized)  
3 if matched:  
4     status = "PASSED"; passed += 1  
5 elif case_timed_out:  
6     status = "TIMEOUT"  
7 elif not case_run_ok:  
8     status = "RUNTIME_ERROR"  
9 else:  
0     status = "WRONG_OUTPUT"
```

Utility script for generating diagnostic/error imagery around evaluator tooling (batch helpers). Not on the hot HTTP path but part of developer ergonomics for debugging OCR failures.

drive_service.py: Drive API scope

```
SCOPES = ['https://www.googleapis.com/auth/drive']  
# Service account JSON preferred; else GOOGLE_API_KEY for public folders only
```

sheets_service.py: entry detection

```
ENTRY_NUMBER_PATTERN = re.compile(  
    r'(\d{4})\s*([A-Za-z]{2,4})\s*(\d{2,5})',  
)  
# Header aliases for entry number, name, marks, comments columns
```

Evaluate page: route and purpose

React route `/evaluate/:evaluationId`

(`frontend-new-design/src/pages/evaluate/index.jsx`) — faculty workflow for one quiz: load answer key, pick ingress (Drive / ZIP / PDF), run OCR pipeline, inspect results, sync/export to Google Sheets, optional Drive rename.

Evaluate UI: layout and panels

Responsive grid (1g:grid-cols-3): left column stacks **Answer Key** panel and **OCR Pipeline** card; wider column for results table, stats, and logs.

Objective vs **code** evaluations swap AnswerKeyPanel vs CodeAnswerKeyPanel.

Shared Tailwind tokens: bg-surface, border-border, text-text-primary, semantic success/error callouts.

Processing mode selector (frontend)

Segmented control with three modes:

Google Drive — folder URL input (`drive.google.com/drive/folders/...`).

ZIP Upload — dashed drop zone, `accept=".zip"`; expects answer key file + sheet images inside archive.

PDF — dashed drop zone `accept=".pdf"`; copy explains one student answer sheet per page, shared OCR/cache semantics with Drive.

ZIP and PDF: client calls (backendService.js)

- ZIP sync: FormData with file; optional query force_reprocess, extract_answer_key; POST /api/process-zip.
- ZIP async: POST /api/process-zip/start → run_id, poll /api/pipeline-runs/{id}/status.
- PDF async: FormData file + evaluation_id (required helper requireEvaluationId); POST /api/batch/process-pdf-optimized/start → processing_id, same poll as Drive batch.
- PDF sync: POST /api/batch/process-pdf-optimized → full PipelineSummary JSON.

Toolbar actions on Evaluate

Sync from Sheet — pull master roster alignment / refresh cues from Google Sheets URL tied to course.

Export to Sheet — POST export pipeline with subsheet + evaluation name; surfaces success message including `studentResponse_*` tab info when created.

Rename Drive files — batch rename after grading when folder URL present.

Live **pipeline log** lines mirror OCR progress (percent, errors, cache hits).

ZIP processing backend (endpoints.py)

POST /api/process-zip: save upload → `_process_zip_archive` extracts to temp tree → walk files into pseudo-Drive dicts (`id=local path`, `name`, `mimeType`, `local_path`).

`drive_service.separate_files` splits **answer key** candidates vs **student sheets** (filename heuristics).

Answer key: first match parsed via `AnswerKeyService`; `extract_answer_key=false` skips ZIP key if server already holds key.

Requires loaded answer key + at least one student sheet or returns 400/404 with actionable detail.

ZIP background starter

POST /api/process-zip/start: persists zip bytes → creates SQLite pipeline run
(source_type="zip", content-aware source_ref) → BackgroundTasks runs
_process_zip_archive_background.

Response: run_id, status_url for UI polling — mirrors long-running Drive batch ergonomics.

PDF split service (pdf_split_service.py)

PyMuPDF (fitz) renders each page to JPEG (default **220 DPI**, quality **90**) → balances checkbox legibility vs payload size.

Full-file SHA-256 pdf_hash for stable identity.

Each page becomes a synthetic “Drive-like” dict: `id = pdf:<hash>:p<n>`, `local_path`, `name`, `_pdf_meta` (`page_number`, `pdf_name`, `hash`).

Same shape lets `_process_sheets_optimized` consume PDF pages without a forked scorer.

PDF batch orchestration (batch_endpoints_optimized.py)

`_run_process_pdf_optimized`: status `splitting_pdf` → `split_pdf_to_pages` → assigns `processing_id`, persists rendered JPEG dir (`pdf_work_dir`) for later **page image** HTTP downloads. Then invokes the **same** optimized multicolumn OCR + cache + scoring path used for Drive folders (including `evaluation_id` scoping). Failure surfaces in `_processing_stats` with `ImportError` hint if PyMuPDF missing.

- POST /api/batch/process-pdf-optimized/start — multipart file, optional evaluation_id form field; streams upload to temp with 1 MiB chunks; returns processing_id, status_url, mapping_url.
- POST /api/batch/process-pdf-optimized — synchronous; returns PipelineSummary or 500 on split/pipeline failure.
- Follow-on: /api/pdf-runs/{id}/mapping, .csv export (optional sheet_url for roster reconciliation), /reconcile, per-page image GET.

Google Sheets roles inside one spreadsheet

The product treats one Google Sheets URL as a **workbook hub**:

Master roster tab — canonical student rows (entry number + name); often named `student_names` when URL omits `#gid`.

Marks tab (per quiz) — created/updated by `update_marks`; per-question columns + total + comments + statistics block.

Student response tab — `studentResponse_{quiz}` with marked answers + answer-key row.

Super Sheet tab — one column per evaluation + cumulative total column for longitudinal grades.

Reading the master sheet (`read_student_list`)

Parses spreadsheet ID + optional gid/tab from URL.

If no tab in URL: prefer tab titled `student_names` (case-insensitive); else first sheet.

Reads grid via `values().get`; `_detect_columns` maps headers to entry / name / marks / comments aliases.

Builds `students` list with physical row indices for later writes.

Side effect: caches roster in `_master_students_cache` for rename fallback.

GET /api/sheets/preview

Non-destructive inspection: returns detected columns, chosen sheet name, full student list + count.
Frontend `backendService.previewSheet(sheetUrl)` powers validation before export/sync.

Writing marks (update_marks)

Reloads master students → smart-matches OCR results to roster rows → sorts rows **confident**, then **fuzzy**, then **unmatched**.

Emits wide table: master entry/name, OCR entry + corrected entry, OCR name, each question score column, SUM formula total column, merged comments (fuzzy labels + name mismatch warnings).

Appends **STATISTICS** block (mean, median, high, low via sheet formulas).

Applies conditional formatting bands for fuzzy/unmatched via `_format_marks_sheet`.

POST /api/export-to-sheets

Body: `ExportToSheetsRequest` (`sheet_url`, optional `results`, `answer_key`).

Query: `subsheet_name`, `evaluation_name` (choose marks tab title).

Resolution order for rows: request body `results` → in-memory `_current_results` → SQLite `students/results` legacy projection.

Steps: (1) `update_marks`; (2) if answer key + tab name — `create_student_response_sheet` as `studentResponse_{tab}`; (3) `update_super_sheet` column for this evaluation.

Returns aggregated summary JSON (per-step errors captured without failing entire export when optional steps fail).

Student response tab layout

`create_student_response_sheet`: columns Name, Entry Number, Q1..Qn from sorted answer-key ids; one row per student with marked text per question.

Trailing blank row then **Answer Key** row (USER_ENTERED); formatter bolds header + key row. Existing tab with same name is cleared then rewritten idempotently.

Super Sheet longitudinal column

Tab name fixed: Super Sheet.

If empty: seed headers Entry Number | Name | {evaluation} | Cumulative Total.

If populated: insert new evaluation column before Cumulative Total, shift cells, recompute row-wise cumulative sums so multiple quizzes accumulate without manual pivoting.

POST /api/export-student-responses

Dedicated path when only the detailed response grid is needed (still requires answer key).
Query `evaluation_name`; optional body overrides for `results` / `answer_key`.
Delegates to `export_responses_to_sheet` (service helper family alongside `create_student_response_sheet`).

Entry-number pattern (sheets_service)

ENTRY_NUMBER_PATTERN expects a four-digit year, a short branch code (letters), and a roll serial with flexible whitespace — used when auto-detecting which master-sheet column holds official entry numbers.

Design takeaway: one workbook, many surfaces

Faculty keep a **single** Spreadsheet ID in course metadata; the backend creates *tabs* for different consumers (roster vs per-quiz marks vs item analysis vs cohort totals). Separating concerns avoids copying CSVs manually while still letting accountants open one familiar Google Sheets URL.

Throughput section roadmap

- ① **Load shaping:** token buckets, round-robin regions, 429 cooldowns, dual OCR + tiebreaker (batch path).
- ② **Streaming pipeline:** bounded concurrency, DB writer queue, two-phase retries, cache tiers.
- ③ **Fuzzy roster logic:** master-sheet vs OCR entry/name (sheets_service).
- ④ **Rename policy:** confident vs fuzzy vs unmatched filenames (drive_rename_service).
- ⑤ **HTTP contracts:** Pydantic bodies, multipart ZIP, polling vs synchronous batch.

Three OCR throughput backends (same Vertex idea, different orchestration)

Module	Role
multi_region_ocr_service.py	Many (region, model) endpoints (Flash / Flash-Lite / Pro); LoadBalancer.get_best_endpoint() scores load + latency + model priority + error rate.
optimized_ocr_service.py	Prefer healthy Flash-Lite with lowest load; fallback endpoints; retries with backoff.
batch_endpoints_optimized.py	Pro-only pool across regions; token bucket RPM ; round-robin ; dual OCR + third majority pass on disagreement.

Load balancer score (multi_region_ocr_service.LoadBalancer)

Lower combined score wins among healthy endpoints under capacity.

```
def score_endpoint(ep):  
    load_factor = ep.current_load / ep.max_concurrent  
    time_factor = ep.avg_response_time / 2.0  
    priority_factor = model_priority.get(ep.model, 4) / 4.0  
    error_rate = ep.total_errors / max(ep.total_requests, 1)  
    return load_factor + time_factor + priority_factor + error_rate
```

If all saturated: pick least-loaded healthy; if none healthy: error path.

Batch OCR endpoint pool (`_ENDPOINT_POOL`)

Nine `gemini-2.5-pro` Vertex regions with RPM budgets (US clusters ≈ 250 RPM each; EU smaller). Combined intent: $\sim 2,250+$ RPM headline capacity; stay on Pro for grading quality (no Flash fallback in this module).

Per-endpoint token bucket (`_TokenBucket.acquire`)

At most rpm tokens per 60s window: spacing $\approx 60.0 / \text{rpm}$ seconds between grants (lock-held schedule).

```
async def acquire(self):
    async with self._lock:
        now = time.monotonic()
        wait = max(0.0, self._next_allowed - now)
        self._next_allowed = max(self._next_allowed, now) + self.interval
    if wait > 0:
        await asyncio.sleep(wait)
```

Round-robin endpoint pick + 429 cooldown

`_pick_endpoint`: skip regions in exponential **cooldown** after HTTP 429; round-robin across remaining pool (global counter + lock).

Cooldown: base 30s, doubles per repeat hit on same region, max 300s; successes clear cooldown counters for that region.

Dual OCR and tiebreaker (batch)

- 1 Two independent Pro OCR passes on **different regions**.
- 2 If answers dicts agree on every question → accept.
- 3 Any disagreement → third pass on fresh region; **majority vote** per question.
- 4 Tiebreaker failure → majority of first two + warning tag in comments.

Streaming pipeline: bounded concurrency

PIPELINE_MAX_CONCURRENT (default 24): semaphore around download+OCR+score so hundreds of files do not spawn hundreds of concurrent coroutines (OOM on 512MB hosts).
download_sem: caps Drive downloads vs OCR slots ($\min(50, \max(8, \text{pipeline_max} * 2))$).

Background DB writer (pipelined commits)

Results enqueue to `asyncio.Queue`; worker flushes SQLite batches (every 20 rows or 2s idle); `None` sentinel ends run.

```
1 async def _db_writer_worker(queue, processing_id):
2     batch = []
3     while True:
4         item = await asyncio.wait_for(queue.get(), timeout=2.0)
5         if item is None:
6             if batch:
7                 await batch_write_student_results(batch, ...)
8                 break
9         batch.append(item)
10        if len(batch) >= 20:
11            await batch_write_student_results(batch, ...)
12            batch = []
```

Two-phase retry strategy (`_process_sheets_optimized`)

Phase 1: all sheets concurrent with bounded retries; empty entry / zero answers queue for phase 2 (does not block cohort).

Phase 2: retries only failures with **fresh OCR** (cache bypass), quieter network after bulk completes.

Cache tiers inside one cache row

Key pattern `sheet:v2:{evaluation_id}:{file_id}` stores JSON blobs:

Full eval hit Valid `evaluation_result`, sane entry, matching `answer_key_hash` → skip vision + scoring.

OCR-only hit Valid `ocr_result` → skip vision, still score.

Stale purge Startup `purge_stale_for_evaluation` removes rows whose hash differs from active key.

evaluation_id guard (cache isolation)

HTTP 400 if missing or literal default — prevents cross-quiz cache bleed.

```
if not evaluation_id or evaluation_id.strip().lower() == "default":  
    raise HTTPException(status_code=400,  
        detail="evaluation_id is required...")
```

Shared HTTP session limits (aiohttp)

One `ClientSession` per batch run with `TCPConnector(limit=256, limit_per_host=32, keepalive_timeout=60)` so concurrent Vertex POSTs reuse connections without exhausting file descriptors.

Fuzzy matching: where it applies

Sheets export correlates OCR rows with a master roster. `SheetsService._smart_match_score` outputs $[0, 1]$ confidence from normalized entry edits + OCR-aware name similarity; exporter writes commentary consumed by `DriveRenameService.classify_match`.

Entry normalization before comparison

Sheet + OCR entries pass `_correct_entry_number_ocr`, strip non-alphanumerics, uppercase — aligns formats like 2025 CSB 1454 vs OCR spacing noise.

Smart match stages (`_smart_match_score`)

- 1 Exact corrected entry: 1.0 if $\text{name_sim} \geq 0.40$ or OCR name empty; 0.98 if entry matches but names differ sharply.
- 2 Year slide: same branch+roll, year within ± 3 , $\text{name_sim} \geq 0.50 \rightarrow 0.95$.
- 3 Levenshtein ≤ 2 on corrected entries + $\text{name_sim} \geq 0.60 \rightarrow$ weighted blend capped 0.89.
- 4 Raw-entry Levenshtein ≤ 2 + $\text{name_sim} \geq 0.50 \rightarrow$ capped 0.88 when corrector mangled tokens.
- 5 Name-only rescue: $\text{name_sim} \geq 0.90 \rightarrow$ up to 0.85 when roll handwriting unusable.

Name similarity fusion (`_name_similarity`)

Word overlap + fuzzy per-word Levenshtein ($\sim 30\%$ edits), substring containment bonus, bigram Jaccard on despaced strings, `difflib.SequenceMatcher` baseline. Weighted sum (words/bigrams/sequence/containment) clipped at 1.0.

Rename classifier (classify_match)

Reads comments + entry:

fuzzy match → fuzzy; not found in master / absent → unmatched; empty/unknown/n/a entry
→ unmatched; else confident.

Drive filename templates

Category	Pattern
confident	{Entry} - {Name}.ext
fuzzy	FUZZY - {Entry} - {Name}.ext
unmatched (OCR text)	UNMATCHED - OCR {entry} - OCR {name}.ext
unmatched (no data)	UNMATCHED_NODATA - {stem}.ext
answer key	ANSWER_KEY - {original}.ext
error file	ERROR - {original}.ext

Duplicate entries append (DUP1), (DUP2) after sanitization.

Sanitization + skip rules

Replace unsafe filename chars; collapse whitespace; truncate length (50 chars default for student tokens).

Optional `skip_already_renamed`: regex detects known prefixes (FUZZY - , UNMATCHED, ANSWER_KEY - , ERROR - , roll-like prefix) to avoid rename churn on re-runs.

Inline rename during batch (`_rename_one`)

When `rename_drive_inline=True`: classify + compute target filename under async lock (duplicate counters); Drive API rename via `asyncio.to_thread`; successful rename updates `file_name/file_id` on the result object for exports.

RenameDriveFilesRequest

```
class RenameDriveFilesRequest(BaseModel):  
    folder_url: str  
    dry_run: bool = False  
    skip_already_renamed: bool = False  
    results: Optional[List[Dict]] = None  
    sheet_url: Optional[str] = None
```

POST /api/rename-drive-files; dry_run returns plan via execute_rename without mutating Drive.

ProcessFolderRequest

```
class ProcessFolderRequest(BaseModel):  
    folder_url: str  
    evaluation_id: Optional[str] = None  
    rename_drive_inline: bool = False  
    group_multiple_pages: bool = False
```

Powers `/api/batch/process-folder-optimized (+ /start)`; server rejects blank/default evaluation ids before work begins.

PipelineSummary

```
class PipelineSummary(BaseModel):  
    total_students_processed: int  
    answer_key_source: str  
    results: List[Any] = []  
    errors: List[Dict] = []  
    processing_stats: Dict = {}
```

Typical synchronous batch completion payload; starters return ids for polling endpoints instead.

Student row schema (StudentResult)

Identity (entry_number, name); aggregates (total_score, max_score, counts); negative_deduction; details list of QuestionResult (marked, correct, result, score); comments carries fuzzy/absent markers; file_name/file_id bind Drive assets.

AnswerKey + AnswerKeyEntry

```
class AnswerKey(BaseModel):
    total_questions: int
    answers: Dict[int, AnswerKeyEntry]
    negative_marking: float = 0.0
    metadata: Dict = {}

class AnswerKeyEntry(BaseModel):
    question_type: str # SMCQ, MMCQ, NCQ
    correct_answer: str
    positive_marks: float = 1.0
    negative_marks: float = 0.0
```

Sheets export models

```
class ExportToSheetsRequest(BaseModel):
    sheet_url: str
    results: Optional[List[Dict]] = None
    answer_key: Optional[Dict] = None

class SheetUpdateSummary(BaseModel):
    updated: int
    not_found: List[str] = []
    name_mismatches: List[Dict] = []
    errors: List[str] = []
```

Full pipeline models

```
class FullPipelineRequest(BaseModel):  
    drive_folder_url: str  
    sheets_url: str  
  
class FullPipelineWithRenameRequest(BaseModel):  
    drive_folder_url: str  
    sheets_url: str  
    rename_files: bool = True  
    dry_run_rename: bool = False
```

Code-eval HTTP models

```
class CodeEvalRequest(BaseModel):  
    answer_key: Dict[str, Any]  
  
class DriveCodeEvalRequest(BaseModel):  
    folder_url: str  
    answer_key: Dict[str, Any]  
  
class CodeEvalResult(BaseModel):  
    total_students_processed: int  
    results: List[Dict[str, Any]]  
    errors: List[str]
```

Multipart vs JSON FastAPI styles

JSON Body models (`ProcessFolderRequest`, `RenameDriveFilesRequest`, `DriveCodeEvalRequest`) with `application/json`.

Multipart ZIP uploads (`UploadFile`) + optional form fields (`answer_key` string).

Query Flags such as `force_reprocess`, `destructive_confirm=true`, cache filters.

Paths `/pipeline-runs/{run_id}`, `/pdf-runs/{id}/mapping`, etc.

Synchronous batch vs background starter

POST /api/batch/process-folder-optimized awaits full pipeline → immediate PipelineSummary.
POST .../start enqueues BackgroundTasks; HTTP returns processing_id.
Poll GET /api/batch/processing-status/{id} for processed counts, cache counters, error tails, partial snapshots.

Representative JSON envelope

Mirrors PipelineSummary: counts, provenance string, results arrays (serialized StudentResult), structured errors, processing_stats (cache_hits, ocr_cache_hits, timing fields).

Failure paths raise HTTPException(detail=...); validation errors include field paths via FastAPI/Pydantic.

Cache DELETE endpoints (examples)

- DELETE /api/cache/purge-evaluation/{evaluation_id}
- DELETE /api/batch/purge-stale-eval-cache
- POST /api/cache/cleanup, POST /api/cache/clear (legacy/global helpers with guards)

PDF run contracts (overview)

Multipart starter → processing_id;
GET /api/pdf-runs/{id}/mapping + CSV export;
POST /api/pdf-runs/{id}/reconcile;
binary page asset routes for QA viewers.

Optimization cheat-sheet (objective batch)

- Token buckets smooth Vertex quota usage ahead of 429 storms.
- Dual OCR + majority vote reduces single-model hallucinations.
- Semaphores cap RAM/FD on shared workers.
- Queue-backed SQLite writer amortizes disk writes during bursts.
- Full-eval cache avoids repeated vision when inputs unchanged.
- Phase-2 retries isolate pathological sheets without blocking everyone else.

OCR-related module map (objective + utilities)

Module	Role
ocr_service.py	Legacy Gemini Flash stream per image.
optimized_ocr_service.py	/ Pooled regional endpoints + load scoring.
multi_region_ocr_service.py	
image_preprocessing.py	Paper isolation + JPEG shrink before POST.
pdf_split_service.py	Multi-page PDF → sheet-shaped dicts + stable hash IDs.
offline_ocr_service.py	Tesseract/Paddle/EasyOCR without cloud.
extract_text_from_images.py	Repo-root script for quick Vertex image tests.
api/batch_endpoints_optimized.py	Orchestrates dual OCR, cache, PDF runs.

Migration 001_result_cache.sql (core DDL)

```
CREATE TABLE IF NOT EXISTS public.result_cache (  
    file_hash          text PRIMARY KEY,  
    file_name          text NOT NULL,  
    ocr_result         jsonb NOT NULL,  
    evaluation_result  jsonb,  
    answer_key_hash    text,  
    created_at         timestampz NOT NULL DEFAULT now(),  
    last_accessed      timestampz NOT NULL DEFAULT now(),  
    access_count       integer NOT NULL DEFAULT 0,  
    result_size        integer NOT NULL DEFAULT 0  
);  
ALTER TABLE public.result_cache ENABLE ROW LEVEL SECURITY;
```

Migration 002_cache_evaluation_scope.sql

```
ALTER TABLE public.result_cache
  ADD COLUMN IF NOT EXISTS evaluation_id text,
  ADD COLUMN IF NOT EXISTS file_id      text;
CREATE INDEX IF NOT EXISTS idx_result_cache_eval_file
  ON public.result_cache(evaluation_id, file_id);
```

Migration 002_pdf_page_map.sql

```
CREATE TABLE IF NOT EXISTS public.pdf_page_map (  
    run_id          text NOT NULL,  
    pdf_hash        text NOT NULL,  
    page_number     integer NOT NULL,  
    entry_number    text,  
    name            text,  
    status          text NOT NULL,  
    total_score     numeric,  
    max_score       numeric,  
    PRIMARY KEY (run_id, page_number)  
);
```

SQLite (database.py) vs Supabase

SQLite: `students`, `submissions`, `results`.

Supabase: `result_cache` (JSONB OCR + eval), scoped by `evaluation_id/file_id`; `pdf_page_map` for PDF reconciliation.

Cache PK: `sheet:v2:{evaluation_id}:{file_id}` consolidates OCR+eval rows.

multi_region_ocr_service: load-aware scoring

```
def score_endpoint(ep: OCREndpoint) -> float:
    load_factor = ep.current_load / ep.max_concurrent
    time_factor = ep.avg_response_time / 2.0
    priority_factor = self.model_priority.get(ep.model, 4) / 4.0
    error_rate = ep.total_errors / max(ep.total_requests, 1)
    return load_factor + time_factor + priority_factor + error_rate
```

optimized_ocr_service: tuning knobs

```
self.max_image_size = (1920, 1920)
self.jpeg_quality = 85
self.optimal_batch_size = 25
self.connection_pool_size = 500
```

Optimized batch OCR policy

Gemini 2.5 Pro only; dual independent OCR; disagree → third majority vote; token-bucket rate limiting; cache hit skips vision + scoring entirely (`batch_endpoints_optimized.py` module docstring).

image_preprocessing.preprocess_for_ocr

```
def preprocess_for_ocr(image_path: str,
                        max_size: Tuple[int, int] = (1920, 1920),
                        jpeg_quality: int = 85,
) -> Tuple[bytes, str]:
    """HSV_paper_crop->resize->JPEG_bytes."""
```

evaluation_service.py: scoring rules

Pure `match_and_score(answer_key, student_answers)` — no LLM.

Normalizes student option tokens; walks each keyed question; supports SMCQ/MMCQ/NCQ with per-question positive/negative marks plus global negative marking fallback.

Treats X, NONE, NA style marks as unattempted (no negative).

Emits `QuestionResult` rows (marked, correct, result, score) aggregated into `StudentResult`.

Parallel-friendly wrappers around scoring (`batch_match_and_score`, optimized answer-key preprocessing) for folder/ZIP throughput — feeds bulk statistics surfaced in batch endpoints.

answer_key_service.py: formats

`extract_answer_key(path, mime)` routes CSV/XLSX/PDF/image parsers; vision path uses Gemini Flash stream via Vertex SA or API key URL when structured parse fails.
Normalizes into `AnswerKey` (answers dict of `AnswerKeyEntry`).
Shared by ZIP ingestion, Drive extracts, and manual uploads.

drive_service.py: responsibilities

Service-account OAuth primary; optional API key mode for public folders.

Folder ID extraction from URLs, recursive listing, MIME-aware downloads, batch rename orchestration, answer-key vs sheet separation heuristics (`separate_files`).

Batch rename batches HTTP calls with retry-friendly chunking (see `batch_rename_files`).

Local engines (**Tesseract**, **PaddleOCR**, **EasyOCR**) plus **ensemble** merge — output shape matches cloud `extract_objective_sheet` so `match_and_score` stays unchanged.
Heavy deps gated behind `requirements-local-ocr.txt`; lazy singletons amortize init cost.

cli.py: headless pipeline

```
python cli.py -drive-folder ... -sheet ...
```

Flags: -mode offline|online, -local-ocr ensemble|paddle|tesseract|easyocr, -tab-name, -no-export, dry-run options.

Flow: enumerate Drive images → offline or Gemini OCR → EvaluationService.match_and_score → optional SheetsService.update_marks.

Useful for CI batches without SPA + complements Docker-hosted API.

Deployment configs

fly.toml: Dockerfile build, internal port 8000, HTTPS, concurrency soft/hard limits, `/health` check, extended idle timeout for long OCR; comments document secrets (`SUPABASE_DB_URL`, base64 SA key).

railway.toml: Nixpacks builder; `pip install -r backend/requirements.txt`; start via `/app/backend/start.sh`.

Tests and utility scripts (repo root)

`backend/smoke_test.py`, `backend/test_phase3.py` — manual sanity harnesses.
`backend/test_heic_ocr.py`, `repo test_heic_ocr.py` — image-format probes.
`extract_text_from_images.py` — quick Vertex vision experiments.
`frontend-new-design/test_supabase_auth.js`, `create_test_user.js` — auth fixtures.
Not wired into unified CI in this deck — document presence for onboarding QA.

Sample artifacts for offline exam bundles:

leetcode_cases_sample.json, testcases_sample.json, testcases_sample.txt, test_expected.txt, test_stdin.txt — illustrate JSON case format consumed by leetcode_mode without hitting live GraphQL.

Pair with cases_file resolver strategy documented earlier.

Documentation artifacts (not slides)

Root README.md, IMPLEMENTATION_SUMMARY.md,
frontend-new-design/IMPLEMENTATION_PLAN.md, REFINED_PLAN.md,
c_code_evaluator/README.md capture historical decisions — treat as companion reading to this PDF.

Browser: /evaluate/:evaluationId

batch_endpoints_optimized.py or Supabase result_cache table

Cross-cutting themes (still this single deck)

Observability (trace IDs), fair grading (blind mode), schema versioning (answer-key JSON), retention/purge on Supabase cache tables.

Future work (selected)

- Signed JWT between FE and BE; org-level quotas.
- Pin LeetCode testcase bundles per exam (`cases_file`) for reproducibility offline.
- Sandboxed execution (Docker/seccomp) instead of raw gcc on shared hosts.
- Golden OCR + golden LeetCode harness regression CI.
- Model routing dashboard (cost vs accuracy).

```
pdflatex presentation3_future_development.tex
```